COMPUTER SYSTEMS AND PLATFORM TECHNOLOGIES

IT0047

FEU ALABANG FEU DILIMAN FEU TECH

Technology Driven by Innovation

MODULE 3

DOS & DEBUG Commands

MODULE 3.2

DEBUG

At the end of the lesson, the student should be able to :

- **Define DEBUG**
- **Identify the most used DEBUG Commands**
- **Write a program with string manipulation using DEBUG**

Debug

DOS offers a debugging utility as one of its external commands. The DEBUG program is used for testing and debugging executable programs. A feature of DEBUG is that it displays all program code and data in hexadecimal format, and any data that you enter into memory must also be in hex format. DEBUG also provides a single-step mode, which allows you to execute a program one instruction at a time, so that you can view the effect of each instruction on memory locations and registers.

Rules of DEBUG command

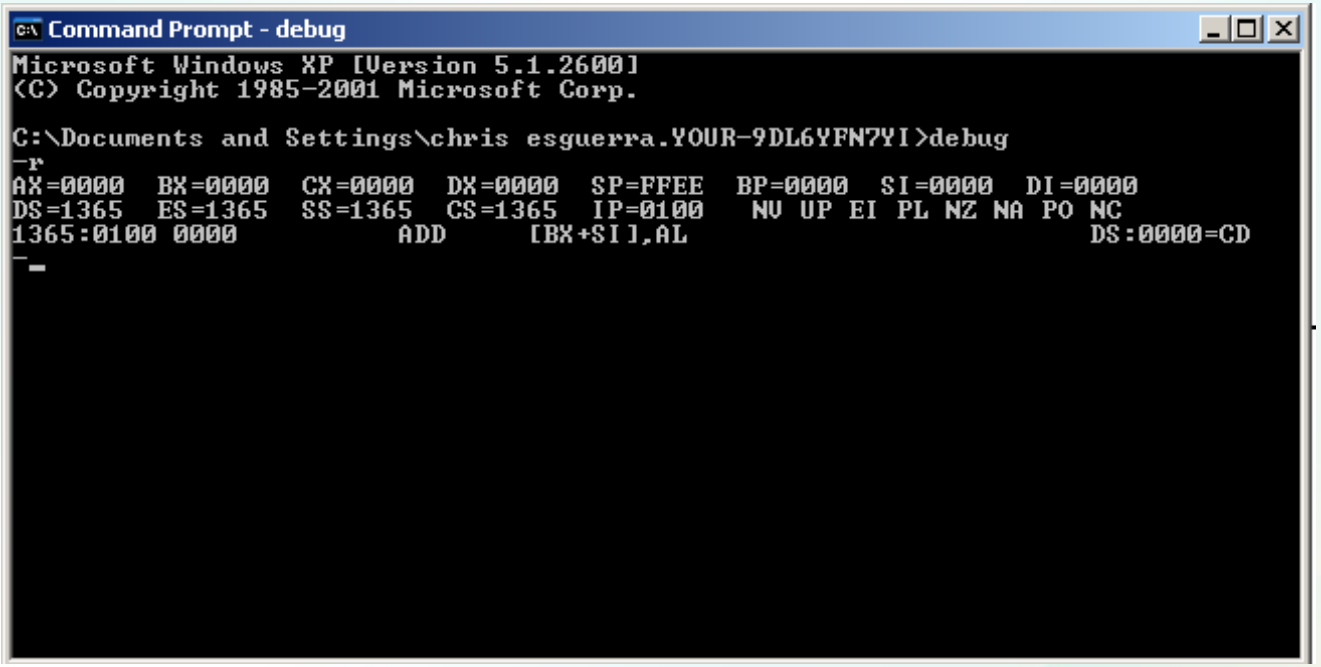
- The prompt from the DEBUG program is a hyphen (-).
- A command is a single letter, usually followed by one or more parameters. Commands become effective only after ENTE00000000000R key is pressed.
- Commands are not case sensitive, can be uppercase or lowercase letter or combination of both.
- Delimiters may separate commands and parameters. Delimiters are only required, however, between two consecutive hexadecimal values.

DEBUG commands

r

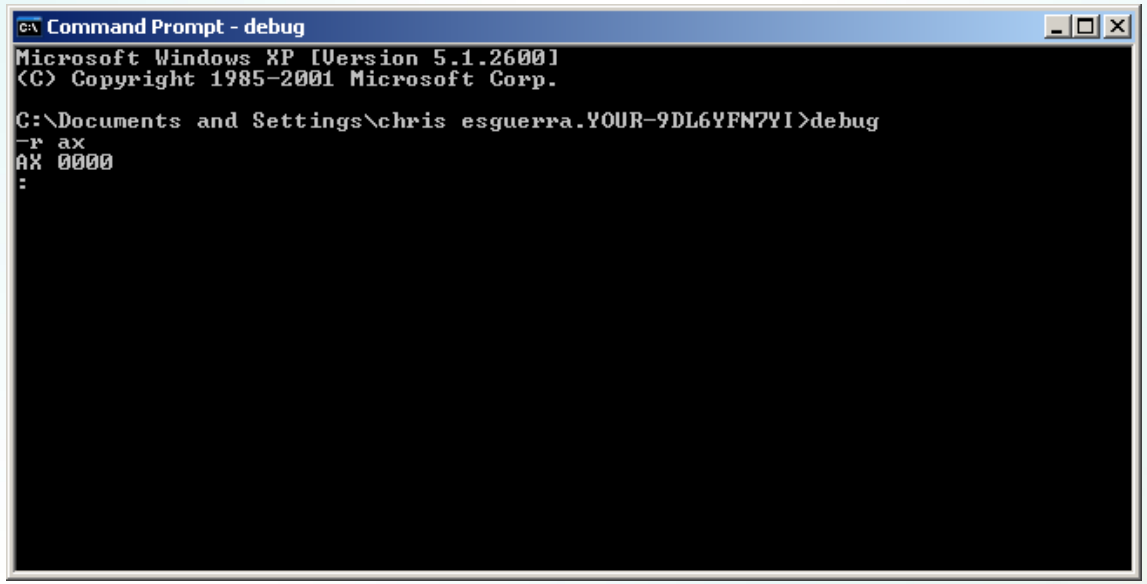
Internal Registers

- **R** - Display the hexadecimal contents of all the register, and the flag register current status and the next instruction to be executed.



Internal Registers

- R <registername> - Display the contents of internal register and with the option of changing those contents.

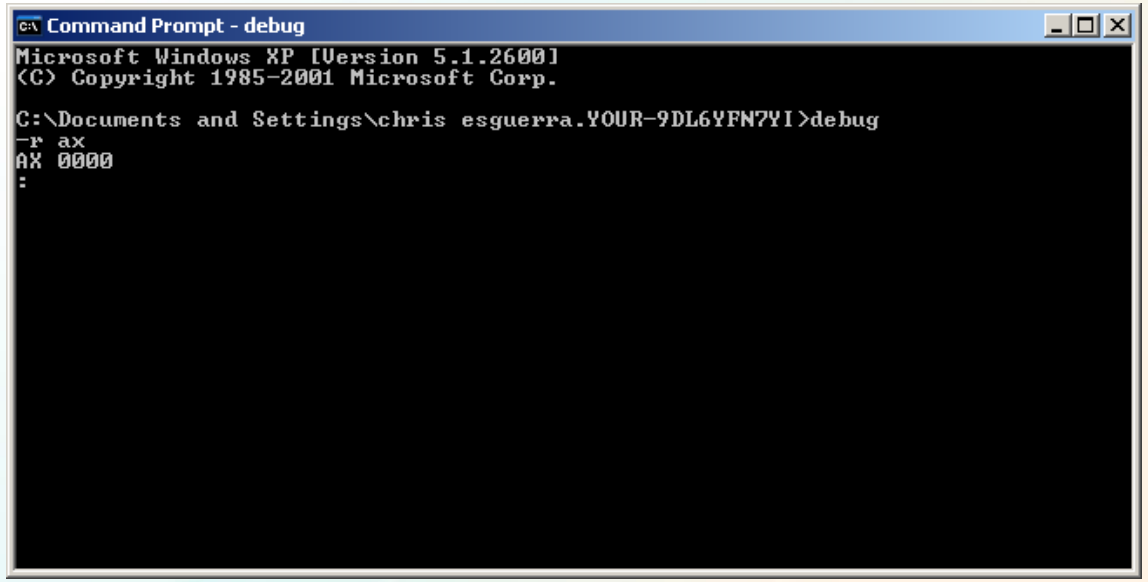


Machine Language Instruction

- The last line of the Register(R) command indicates the location in memory of the next instruction to be executed and its machine mnemonic code (unassembled) codesThe E (Enter) Command displays and allows modification of bytes in a sequential manner.

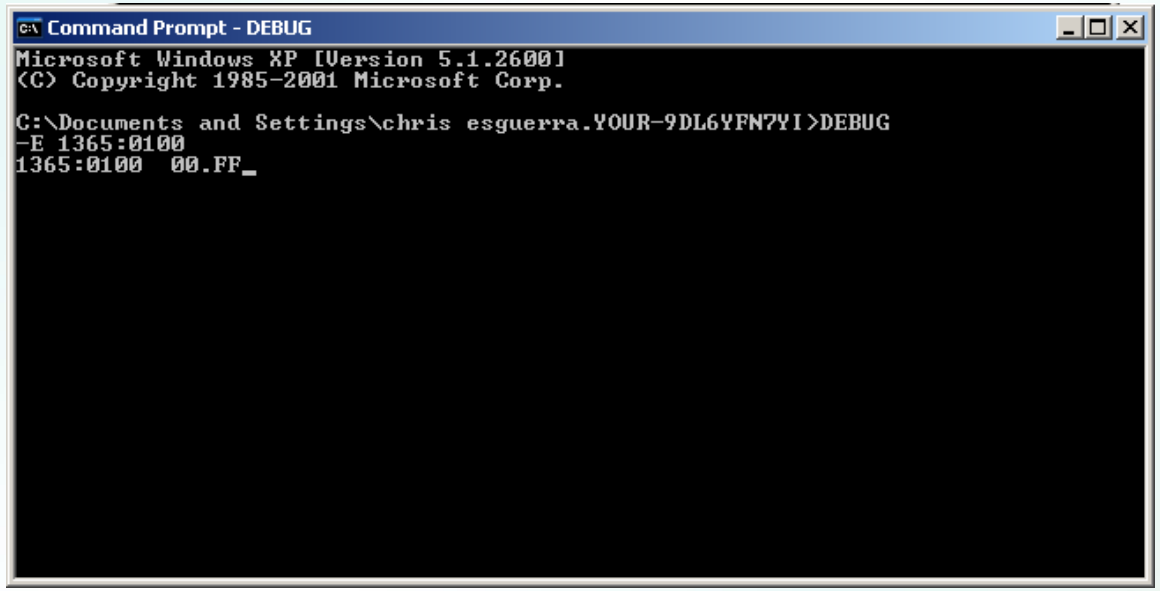
Format: E <address>

Note: Address should contain the code segment and instruction pointer where you want to modify.



Machine Language Instruction

To change the content of the address specified above, from 19h to FFh, enter FF after the dot (.).



Mnemonic Codes

- The U (unassembled) command translates the contents of the memory into its equivalent mnemonic code assembler – like statements and displays their addresses.

  Format: U <address>

```
-u 1365:0100
1365:0100 01C0          ADD     AX,AX
1365:0102 01D8          ADD     AX,BX
1365:0104 01C8          ADD     AX,CX
1365:0106 01D0          ADD     AX,DX
1365:0108 29C0          SUB     AX,AX
```

Unassembled from the first address to last address specified in Enter Command activity and translate each machine code into its equivalent mnemonic code. Write down on the data sheet the results of this activity.

Program Execution

Writing Down Program in DEBUG Environment

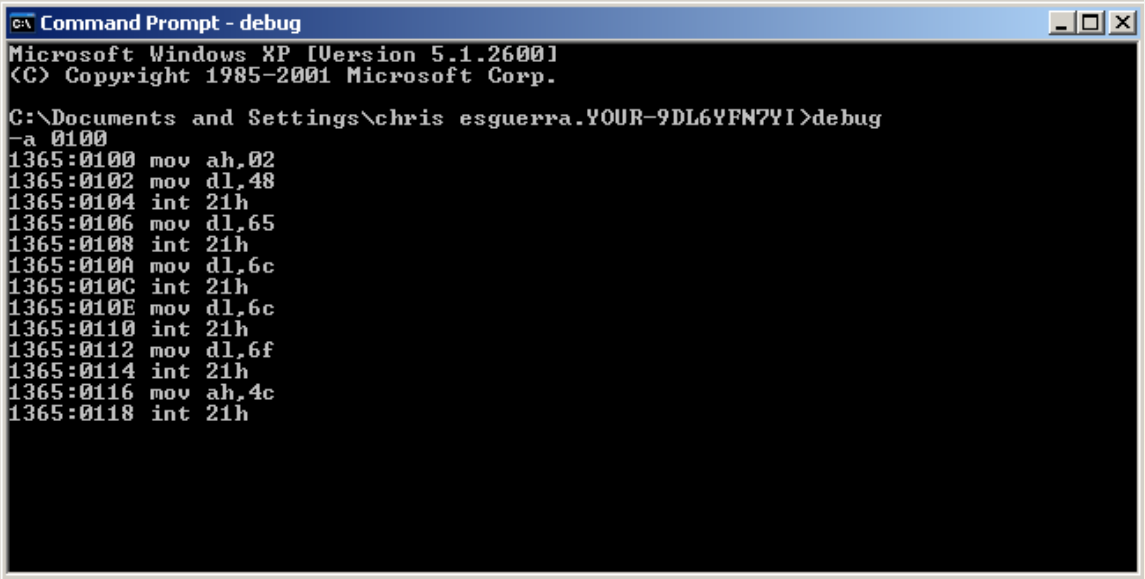
The A instructions.

(Assemble) command lets you enter mnemonic code assembly language

Format: A <address>

- Assume for the address that will serve as the offset address.

Assemble the following code into memory location 1365:0100.



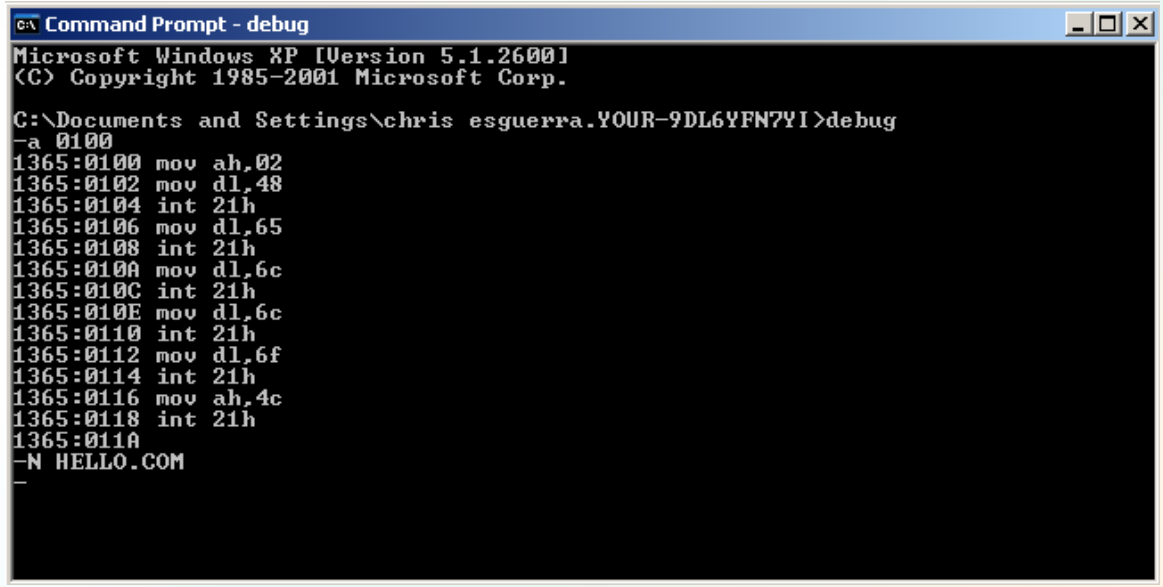
Program Execution

Saving the Program into the Disk

To write a file, you must first initialize the name of the file.

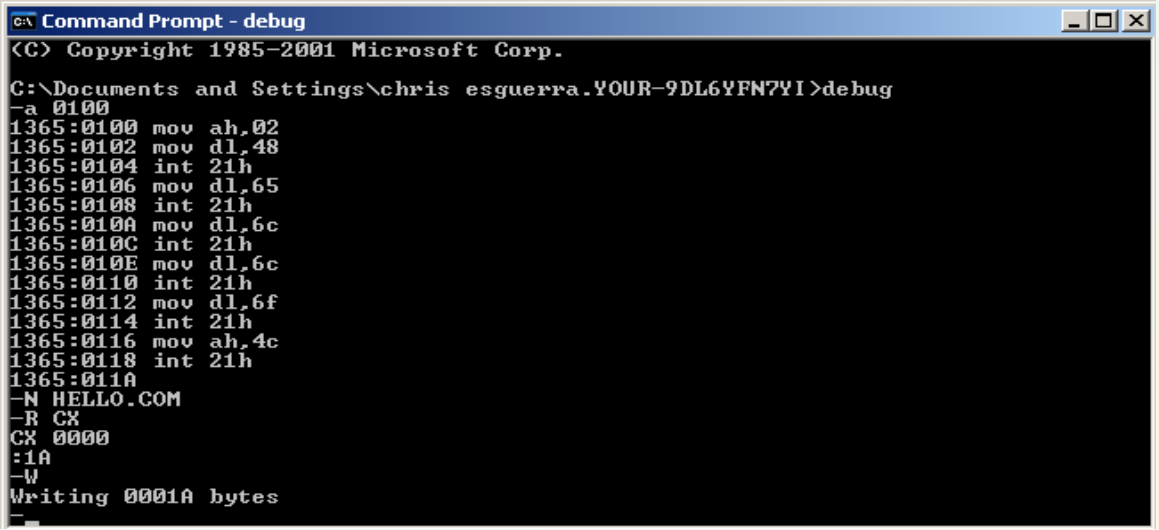
Format: N <filename>.<ext>

The N command initializes a filename in memory before using load or write command.



Program Execution

In writing the program to the disk, computer for the byte consumed by the program then, store it in CX (Count) Register. To compute, subtract the starting address to the last address covered by the program plus 2.

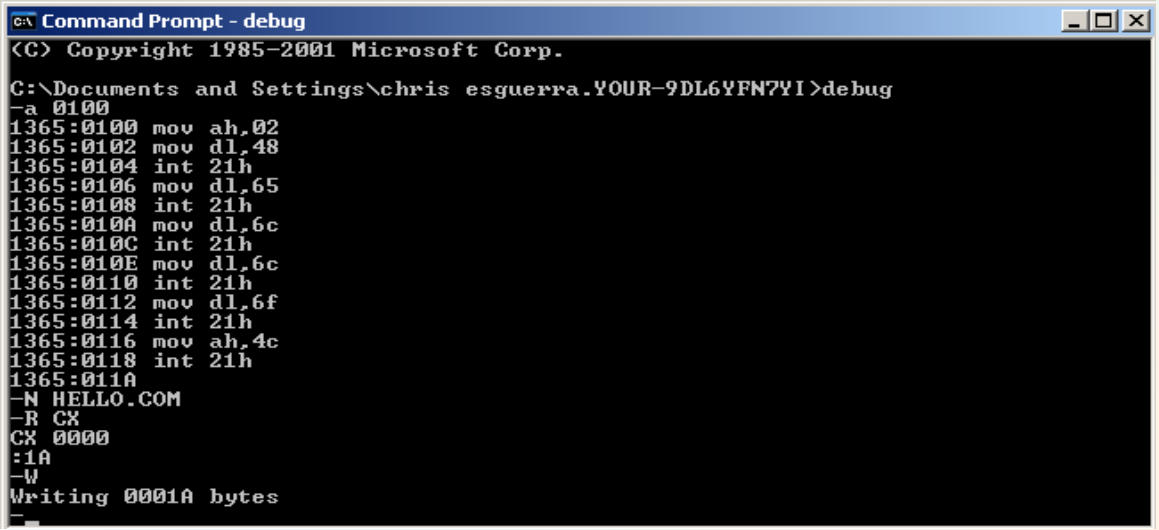


Note: Computation is in hexadecimal Arithmetic Operation

Consumed byte = (0118 – 0100) + 2

Program Execution

In writing the program to the disk, computer for the byte consumed by the program then, store it in CX (Count) Register. To compute, subtract the starting address to the last address covered by the program plus 2.



Note: Computation is in hexadecimal Arithmetic Operation

Consumed byte = (0118 – 0100) + 2

The W (write) command writes a block of memory to a file.

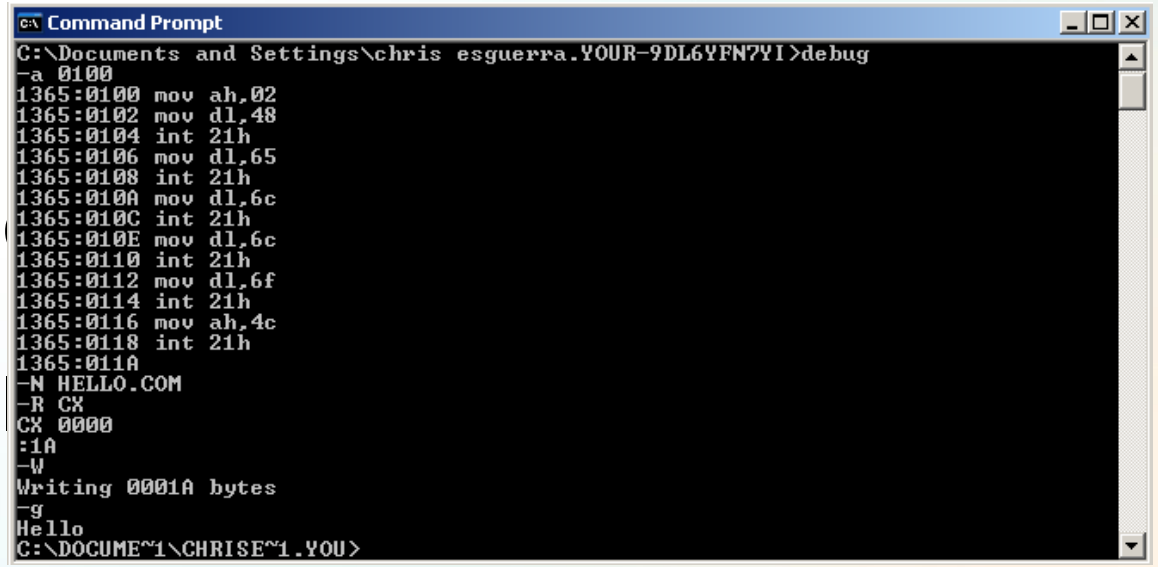
Program Execution

Executing the Program

Using the G (GO) command, it will execute the assembled program.

Format: G

The T (trace) command executes one more instructions starting with the instruction at CS:IP. The contents of all register and flags after each execution are also displayed.



Format: T

DOS FUNCTION CALLS

To make the task of programming in Assembly Programming easier, DOS provides a set of instruction for character device I/O, file management, memory management, data and time functions, execution of the other programs, and others. This set of instructions is assigned as interrupt vector 21h, and each function can be called upon by the INT 21H instruction.

DOS FUNCTION CALLS

Using DOS Function

DOS Function INT 21H (“Function Dispatcher”) requires the function number in register AH on entry only. Most of the function calls require input to be passed to them in the other register.

1. Set the proper register values required.
2. Place the function number in AH.
3. Issue interrupt type 21H.

TRADITIONAL CHARACTER DEVICE INPUT AND OUTPUT

Function : Direct Console Input without Echo
Purpose : Waits for a character to be read from the keyboard then returns with the character in AL.
On Entry : AH = 07H
On Exit : Character read from the keyboard.
Function : Character Output
Purpose : Outputs a character to the standard output device, usually monitor screen. On Entry : AH = 02H
DL = ASCII code of the character to be output
On Exit : NONE

TRADITIONAL CHARACTER DEVICE INPUT AND OUTPUT

Function : Character Input with Echo
Purpose : Waits for a character to be read from the keyboard, then echoes it to the screen and return with the character in AL.
On Entry : AH = 01H
On Exit : AL = Character read from the keyboard

String Manipulation (DEBUG)

Displaying strings on the screen and reading them from the keyboard is entirely different from displaying or reading characters. ASCII codes of all characters are only a byte, which would easily fit inside any of the internal registers of the 8088. A function that displays a character only needs the character to be displayed to be put in the DL register, e.a. function 02h. Similarly, a function that reads a character from the keyboard can easily get the character read in the register AL, e.a. functions 01h and 07h.

String Manipulation (DEBUG)

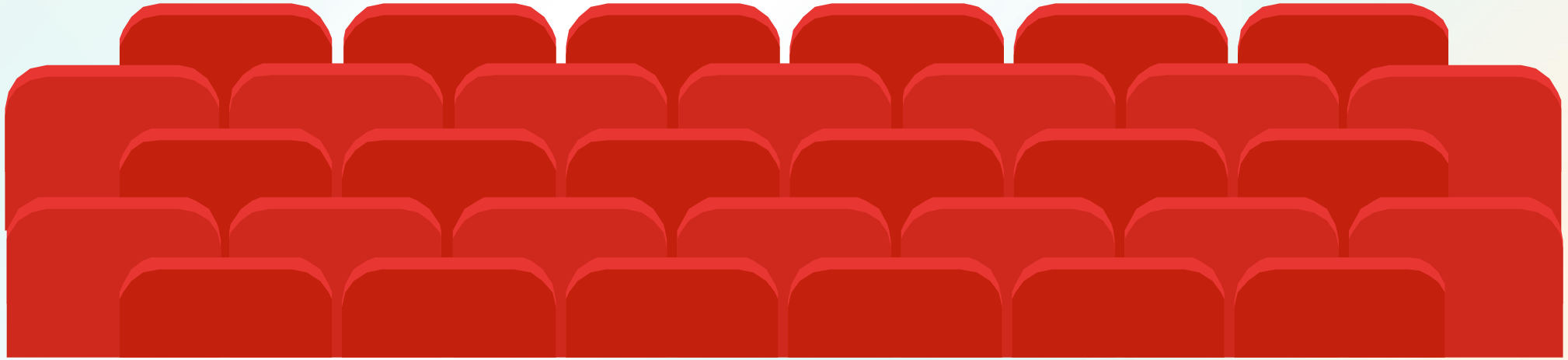
A string is usually more than one character length. This leaves us with the question: “Where then can a function that displays a string put the string to be displayed?” The answer is
– the memory. Since string’s can’t usually fit in a register, strings are then placed in the memory then pass the address of the string in memory to two of the registers – the segment address and offset address in DX.

String Manipulation (DEBUG)

Function : String Output
Purpose : Sends a string of characters to the standard output device On Entry : AH = 09h
DS = Segment address of the first character of the string DX = Offset address of the first character of the string
On Exit : NONE

Note: Function 09h displays a string of characters starting with the first character (address in DS:DX) output, but not including the character “$” (ASCII -24h)

Q&A SESSION

ASK ANY QUESTION RELATED TO OUR TOPIC FOR TODAY.

THANK YOU